An Amateur's Alternative to PRESTO for Pulsar Analysis
Peter W East

Abstract
PRESTO is the go-to software package for amateurs interested in analysing pulsar observations to recover and verify true pulsar acquisition in a typical RFI environment [1]. Originally produced to find new pulsars in extensive data files recorded by professional large aperture telescopes, PRESTO employs a fast power detection algorithm that limits its performance against recovered pulsar peak signal-to-noise power ratios lower than about eight to one. The PRESTO pulse detection measure employs the reduced chi-square statistic which senses pulsar presence simply from the pulse mean power rather than peak signal power-to-noise power ratio (SNR), making it faster but less sensitive to low duty cycle pulsar emissions. To make pulsar detection available to a wider group of amateurs with space-limited antenna apertures, an alternative software package has been developed to simulate the PRESTO pulsar recognition features by using the signal-to-noise ratio pulse presence statistic.

Introduction
Amateur pulsar detection is a 3-stage process involving directing the radio telescope antenna, collecting data over a long period and finally processing the data based on the folding algorithm to sum consecutive weak pulses to magnify the pulsar signal. Pulsar names conveniently include their right ascension and elevation coordinates for ease of location. After describing the basic requirements of a pulsar radio telescope, the paper concentrates on the software processing requirements to recover pulsar signal properties for detection verification.

Basic Pulsar Radio Telescope
Figure 1 shows basic a starter Radio Telescope for detecting the loudest pulsars and proven to give consistent results [2].

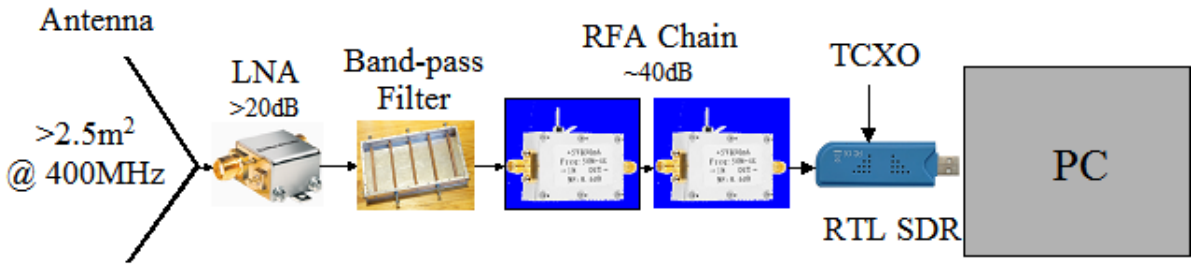

Figure 1. Basic Pulsar Radio Telescope

The 400MHz frequency band is a good choice; amateurs have been successful in this band as the power here is some 7 times that at the hydrogen line frequency of 1420.4MHz
Here, an antenna of 2.5 square metres capture area, feeds a closely coupled low noise amplifier (LNA) of less than 0.5dB noise figure, a band defining filter to minimise out-of-band RF interference; a couple more amplifiers to raise the signal level to drive the RTL2832U defined radio and a personal computer to collect and pre-process the sampled data.
A temperature compensated crystal oscillator is shown here as it important that the RTL2832U SDR is accurately clocked to better match the pulsar rotation stability. An alternative is to lock the RTL local oscillator with a GPS disciplined oscillator or a rubidium standard.
The PC software processing performs the tasks of data recording, data pre-processing and recovering the pulsar power in two or three stages as discussed later.
A quality LNA is required with about 20dB gain and less than 0.5dB noise figure. The precise band pass filter frequency around the 400MHz band is best chosen after identifying a local RFI free band.
The digital TV dongle based on the Realtek RTL2832U is a popular SDR, since with the free Windows driver and Osmocom recording software, the dongle allows transferring the raw data samples to the host computer for recording and analysis.

Pulsar Software Key Processes

The PC software processing performs the tasks of data recording and recovering the pulsar power in two or more stages and there are various versions for amateurs.

The main processing functions required are,

0. Input pulsar target information.
1. Data recording.
2. Channelising in frequency for later de-dispersion.
3. Power detection.
4. Averaging to improve the SNR and reduce the video bandwidth.
5. RFI reduction.
6. De-dispersion.
7. Folding to sum pulse periods to further reduce the video bandwidth and increase the SNR.
8. Pulsar parameter analysis for detection verification.
9. Data Plotting.

The observation pulsar topocentric rotation period is required (software requirement 0) and there are a number of other intermediate functions necessary to identify pulsar properties to prove verification.

Currently Available Software Used by Amateurs[3,4,5,6,7,8,9]

Table 1 lists some useful sources of pulsar software with the software functions covered as numbered above.

Table 1. Available Software Packages

Originator	Description	Processes Covered
M. Klaassen	WINDOWS, 3pt RTL SDR analysis tools http://parac.eu/projectmk17b.htm	2,3,4,6,7,8
A. Dell'Immagine, G. Dell'Immagine	LINUX USB Boot https://github.com/gio54321/pulsar-distro-guide	1,2,3,4.
M. Leech:	LINUX/WINDOWS, GNURadio flowchart https://github.com/ccera-astro/pulsar_filterbank	1,2,3,4
Professional	rtl_sdr.exe: Osmocom https://osmocom.org/projects/rtl-sdr/wiki/Rtl-sdr GNURadio: https://www.gnuradio.org/ TEMPO: pulsarastronomy.net/pulsar/software/tempo/ PRESTO: https://www.cv.nrao.edu/~sransom/presto/ PSRCHIVE, pulsarastronomy.net/pulsar/software SIGPROC, pulsarastronomy.net/pulsar/software DSPSR: pulsarastronomy.net/pulsar/software ATNF: https://www.atnf.csiro.au/research/pulsar/psrcat/	1. 1.2.3,4 0. 5,6,7,8 5,6,7,8 5,6,7,8 5,6,7,8 0.

Michiel Klaassen's 3pt software runs in WINDOWS, takes recorded RTL-SDR data and carries out a comprehensive analysis and produces a professional Python display.

Andrea Dell'Immagine and his son Giorgio have developed pulsar recording software and guide that boots a pulsar-friendly LINUX from a USB stick. It controls some common SDRs to record data; it channelises, detects and averages the data and outputs it in the professional's filterbank file format suitable for analysis by most of the relevant professional software.

Marcus Leech, from the Canadian Centre for Experimental Radio Astronomy has developed a GNURadio pulsar package that controls several SDR types and outputs channelised, detected and downsampled data in filterbank file format, again for PRESTO or other professional package analysis.

GNURadio is a free, easy to use, graphical digital signal processing package for all receiver digital signal processing needs. There are both Windows and Linux versions that are well-documented and accepts/controls a range of SDR's..

TEMPO is a pulsar timing prediction package and provides information required for finding the observed pulsar rotation period at the time and local coordinates of the observation. An alternative Python code using the Astropy module is decribed in Appendix 1.

PRESTO is the professional's software for extracting pulsars in recorded data and this is only available for the Linux operating system but is used by most radio astronomy amateurs.
The Australian Telescope National Facility (ATNF) data base holds all known pulsar parameters.
PRESTO's popularity with amateurs arises from its powerful algorithms for RFI mitigation in both frequency and time and the detailed searches it carries out to result in visual verification plots identifying period, period rate and dispersion measure agreement with the known pulsar characteristics.

Alternative Software Targeting Low SNR Observations [10]

The driver for this investigation was to trade analysis and verification processing speed against being able to verify pulsar detection by simulating the PRESTO processes but using the more sensitive SNR power recognition statistic [10]. This was to improve recognition and verification of low SNR acquisitions from weaker pulsars or small aperture receiving systems.
As an extra feature, to optimise the performance against strongly scintillating pulsars, the pre-processed data file is scanned with a variable width window to find the best data range producing the strongest SNR pulse profile.
A major analysis speed improvement was made by synchronously compressing the data into just 128 periods (see Appendix A6), enabling the search and analysis processes to be carried out speedily with no loss of information.

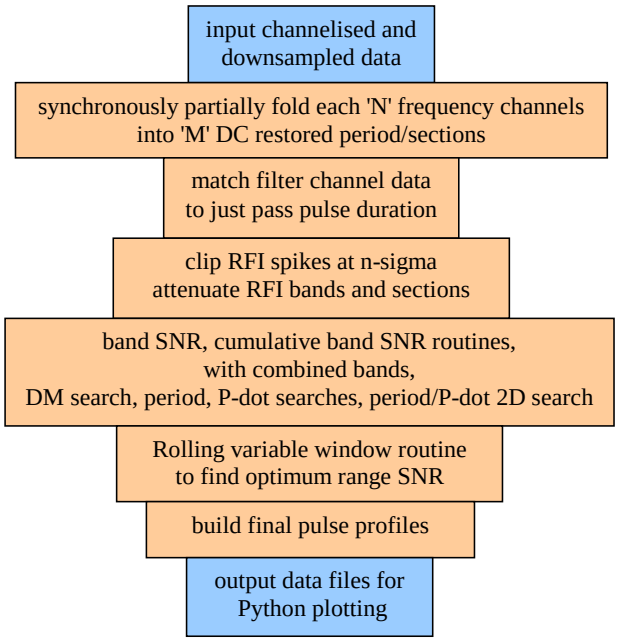

Figure 2. *pulsar_det_an.exe* software algorithm functions

Figure 2 highlights the key processes carried out by the new software, *pulsar_det_an.ex*e (Appendix A4). Following the partial folding process (see Appendix 6) on all frequency channels, the outputs are DC restored to remove receiver low frequency drift, the data is then filtered by a convolution process to optimise detection sensitivity by matching the data bandwidth to faithfully pass the expected pulsar pulse shape. Both RF and video spectra are recorded. Facilities are included to attenuate frequency bands and period sections if RFI identified and also to clip RFI spikes as a function of section SNR. Band and section peak and target SNR measures together with cumulative SNR with time and section number are also stored in files for post processing. Dispersion measure, period, P-dot and period/P-dot 2D searches are also recorded for peak position validation [?].
An advantage of partially folding the data into a number of sections is that sequential section groups can be combined and the group showing the highest target SNR identified (see Appendix 7). This is useful for long drift-scan observations against randomly scintillation pulsar sources. In some cases it can prove indicative of the antenna beam-shape; an extra pulsar target verifier. This search process is carried out in two stages. Firstly, a window of a set number of sections is scanned over the whole

section range and plotting the maximum value it finds. This is carried out with all possible window sizes. Plotting the SNR peak against window size we find the maximum target SNR together with the relevant section range size. By scanning the optimum window size again across the section range and measuring the window SNR, the peak range and corresponding best SNR is identified. Appendix A7 illustrates an example.
A number of text files are produced including the search and folded profiles in the local directory for processing by the display program *pul_plot py* to produce a summary plot indicating the pulsar verification parameters in a form similar to that produced by PRESTO (see Appendix A5).

Amateur Pulsar Package [11]
The full data acquisition, data analysis and display processes for the RTL and other SDRs is summarised in Figure 3.

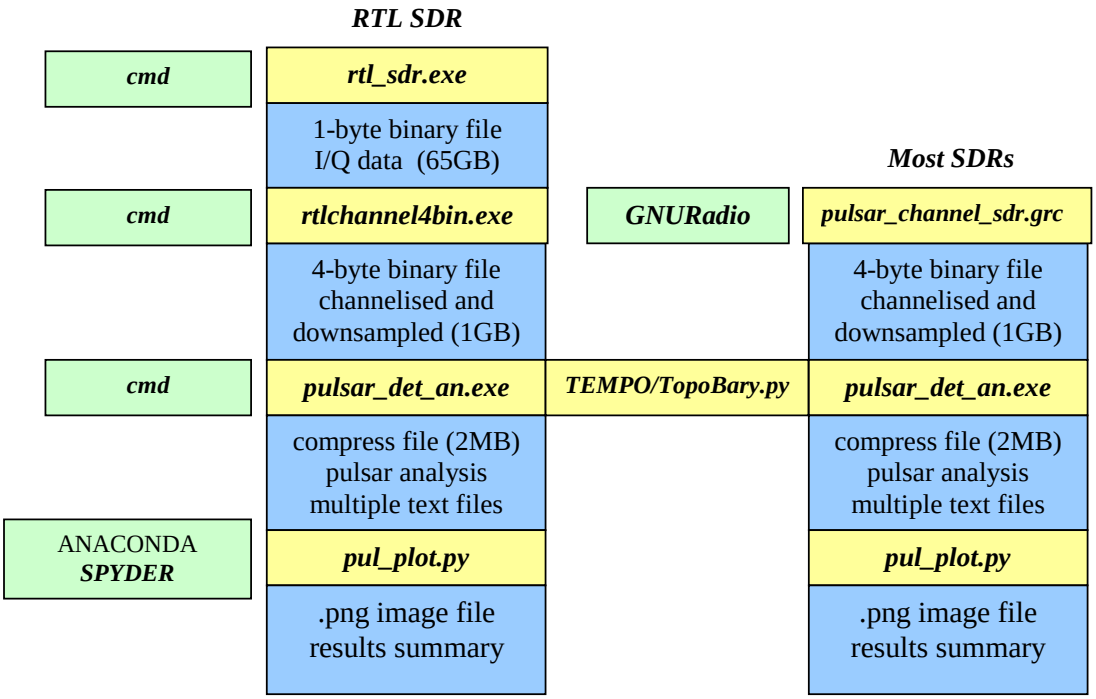

Figure 3. Software Processes and Data Collection/Processing Sequence

The new software source code, written in standard ANSI C, is provided and can be compiled with free compilers, MinGW in Windows or GCC in Linux for 32 or 64-bit computers as required.
The executing files produced are run in Windows command prompt or Linux terminals together with specified command line instructions.
For the RTL SDR, Osmocom have made available the *rtl_sdr.exe* program to collect raw I/Q data which is in a 1-byte binary form. PRESTO requires the raw data to be split into a number of frequency bands (channelised), downsampled and converted to filterbank .fil format. For *puldsar_det_an.exe*, the C-program *retchannel4bin.exe* (see Appendix A2) carries out this function but outputs the data in a 4-byte binary form suitable for fast processing by *pulsar_det_an.ex*e.
An alternative shown in Figure 3 is to use GNURadio to carry out both pre-processing requirements and output the data in a compatible form. In fact, both the Andrea Dell'Immagine and Marcus Leech packages of Table 1 use GNURadio to this end [7]. A basic GNURadio .grc graphical display for the Airspy R2 SDR is shown in Appendix A3.
The professional TEMPO software computes the observed pulsar rotation frequency and period at the observation time, latitude and longitude. Here, the Python program TopoBary.py (see Appendix A1) can also carry out this requirement if the Astropy module is installed with Python 3. Downloading the free ANACONDA package includes Astropy and also includes the SPYDER IDE, useful for running Python programs and is recommended [12].

*pulsar_det_an.ex*e is run in a command terminal and analysis input parameters are presented in a command line, defined as,

pulsar_det_an.exe <binfile> <N-point FFT> <data clock (ms))> <Topocentric period (ms)> <No: fold sections><No. FFT bins><ATNF pulse width><ATNF DM><ppm period offset><spike threshold, x sigma><ppm range factor><RF band (MHz)><RF Centre (MHz)><roll average No.><start section><end section>

An example using the Airspy R2/GnuRadio approach is:-
*pulsar_det_an.ex*e airntmon1.bin 16 2 714.492518 128 1024 6.5 26.7 0.0 6 1 10 611 37 0 127

where,
airntmon1.bin is the input data file generated by GNU Radio as a 32-bit binary file of detected data samples, and,

16 is the number of Fast Fourier Transform (FFT) channels/bands.
2 is the set data clock in ms.
714.492518 is the TEMPO topocentric period for the observation site and recording UTC
128 and *1024* are the numbers of compressed file sections and fold bins respectively - both numbers need to be powers of 2.
6.5 is the target pulsar pulse width.
26.7 is the target pulsar dispersion measure
0.0 is the period correction offset in ppm.
6 is the rms/sigma factor to limit random RFI spikes.
1 is the period search range divisor.
10 is the SDR RF bandwidth in MHz
611 is the RF center frequency
37 is the rolling average section range.
0 is the file 1st section start selected.
127 is the 128th section end selected.

For information, the GNURadio pre-processed file, *airntmon1.bin*, size is 525MB and results from 168Gs of raw Airspy R2 data samples at 10MHz.
Figure 4 shows the results of the analysis of *airntmon1.bin* as presented by a .png image by running *pul_pot.py* in Spyder on the *pulsar_det_an* output text files.

Table 2 lists a summary of the data analysis defining input and final results. Whilst folding the full data results in a final SNR of 4.7:1, but the optimum section range upgrades this to nearer 8.2:1.
Dotted plots are expected theoretical results assuming a Gaussian-shaped pulse of the expected pulse width.
The plots in Figure 4 from left to right and top to bottom are,
Plot 1. Pulse profiles; blue: folding the whole file, SNR=4.7:1, red: best section group result, SR = 8.2:1.
Plot 2. Dispersion measure scan showing a peak around the expected DM of 26.7 (blue indicator). red: all data search result; red dotted curve: theoretical noiseless prediction. black crosses: peak bin number.
Plot 3. Period (red) and period rate(blue) search plots clearly peaking as expected at zero offset. red and blue dotted curves: theoretical noiseless predictions.
Plot 4. The next plot is the frequency band scan results showing both a cumulative increase (red) and target frequency scintillation (blue).green crosses: band peak SNR, black crosses: peak in number.
Plot 5. Cumulative section target SNR, green: section peak, orange: target peak showing steady growth in SNR, black crosses: cumulative peak bin number.
Plot 6 Orange: optimum section range search showing a peak with about 40 consecutive sections, magenta curve indicates peak SNR occurs when the 40 sections are centred on section 50, red dots indicate section(red) and target(blue) SNR.
Plot 7 Shows the section and target SNRs and Plot 8 shows the data spectrum.

Table 2. *pulsar_det_an.ex*e airntmon1.bin 16 2 714.492518 128 1024 6.5 26.7 0.1 6 1 10 611 37 0 127

Pulsar Data	Number of pulsar periods = 23512
Input Data file: = airntmon1.bin	P-dot factor = 10000
Thres: 6 sigma Period Range Multiplier = 1.00	ppm adjustment = 0.00
Pulsar Period = 714.492518 ms	Max Pulse ppm drift = 0.00 ms
Data clock= 2.00 ms	Compression Ratio = 183.70
Pulsar Pulse Width = 6.50 ms	Period Search Range = -25.00 ppm to 25.00 ppm
Pulsar Dispersion Measure = 26.70	P-dot Search Range = -21.27 ppm/10000 to 21.27 ppm/10000
Input file bytes = 268800000	Rolling Average Number = 37
RF Centre = 611.0 MHz, RF Bandwidth = 10.0 MHz	**Start section = 0**
DM Band Delay = 9.716ms, No. Delay bins = 13.924	**End section = 127**
No. Data Samples = 134400000	Attenuated Bands:
No: FFT Bands = 16	Noise Substitute Sections:
Input Data per FFT Band = 8400000	Max bin = 632 Max SNR = 4.70
No. of Output Fold Sections = 128 ; No. bins = 1024	
No. of Output samples = 131072	Best Section Range SNR = 8.23 Section Centre 50
Working file duration = 16800 secs	Best Section Range 38

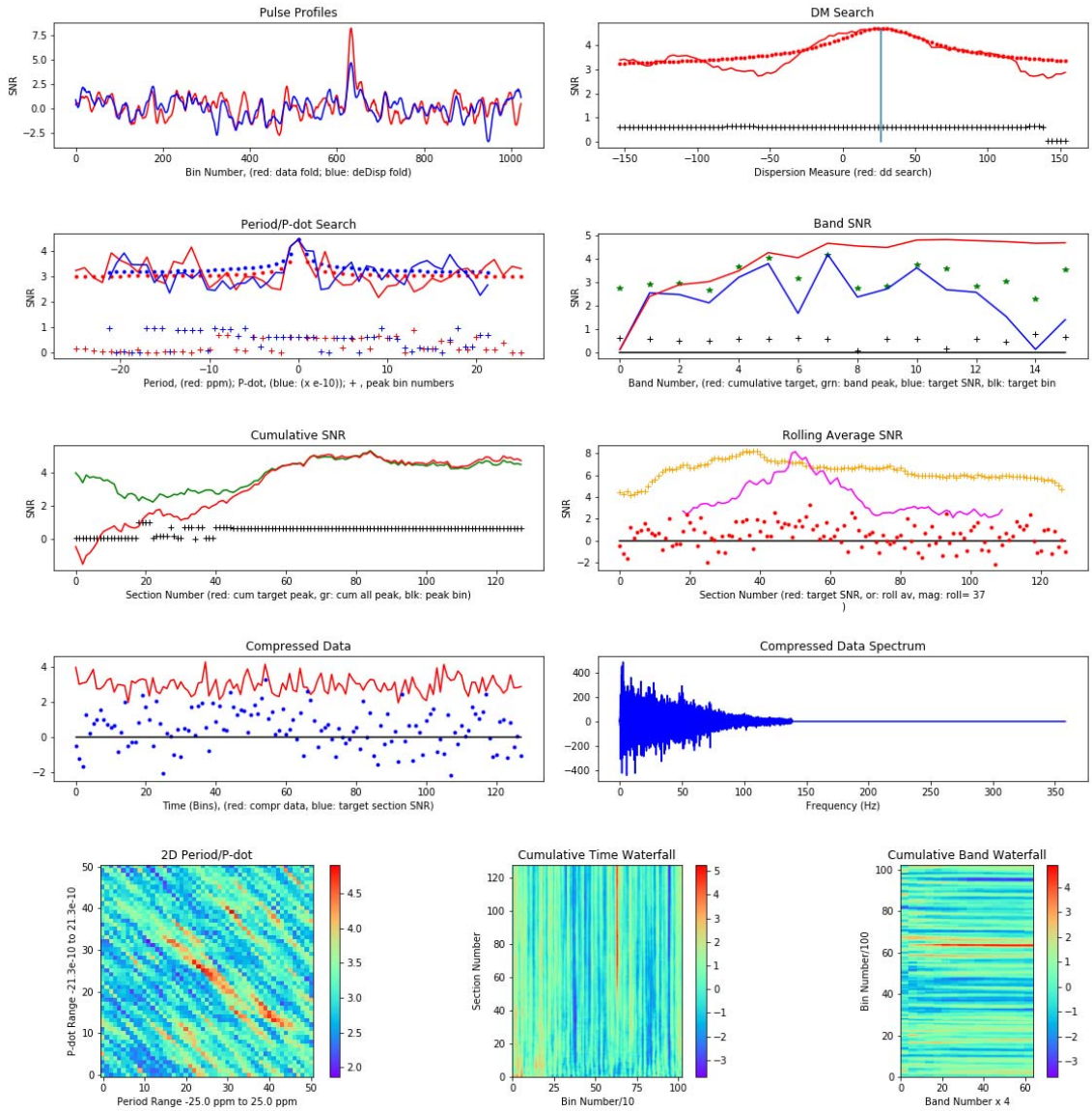

Figure 4 *pul_plot.py* pulsar data presentation -full data analysis

Figure 5 plots the result from just using the data from the best section range identified by analysing the previous full data file range. The revised command instruction for Table 3 is:-

Table 3 *pulsar_det_an.ex*e airntmon1.bin 16 2 714.492518 128 1024 6.5 26.7 0.1 6 1 10 611 37 31 69

Pulsar Data Input Data file: = airntmon1.bin Thres: 6 sigma Period Range Multiplier = 1.00 Pulsar Period = 714.492518 ms Data clock= 2.00 ms Pulsar Pulse Width = 6.50 ms Pulsar Dispersion Measure = 26.70 Input file bytes = 268800000 RF Centre = 611.0 MHz, RF Bandwidth = 10.0 MHz DM Band Delay = 9.716ms, No. Delay bins = 13.924 No. Data Samples = 134400000 No: FFT Bands = 16 Input Data per FFT Band = 8400000 No. of Output Fold Sections = 128 ; No. bins = 1024 No. of Output samples = 131072 Working file duration = 5119 secs	Number of pulsar periods = 7163 P-dot factor = 1000 ppm adjustment = 0.00 Max Pulse ppm drift = 0.00 ms Compression Ratio = 183.70 Period Search Range = -25.00 ppm to 25.00 ppm P-dot Search Range = -6.98 ppm/1000 to 6.98 ppm/1000 Rolling Average Number = 37 **Start section = 31** **End section = 69** Attenuated Bands: Noise Substitute Sections: Max bin = 633 Max SNR = 7.57 Best Section Range SNR = 8.22 Section Centre 63 Best Section Range 119

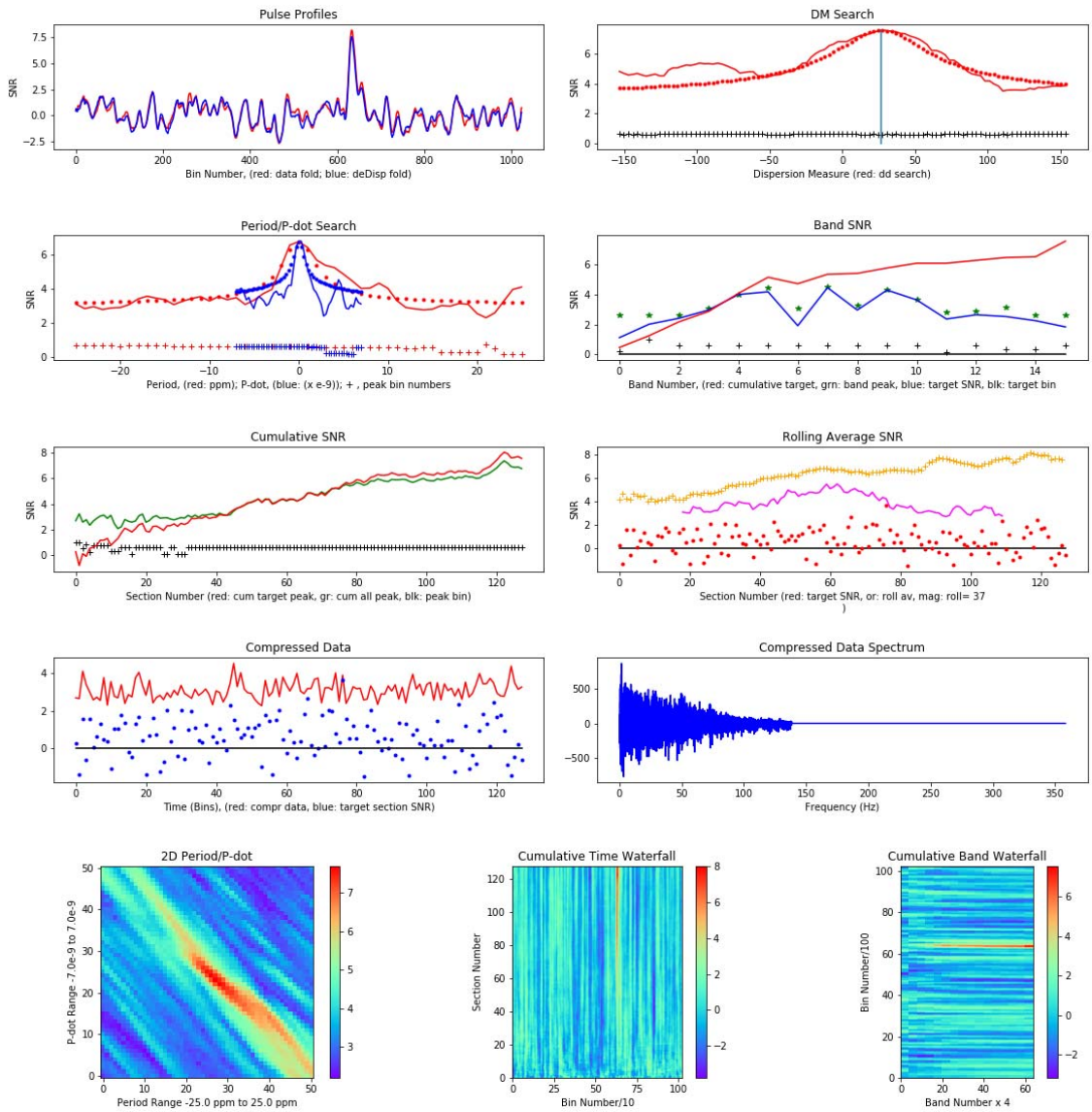

Figure 5 *pul_plot.py* pulsar data presentation - best section range analysis

A cursory comparison of Figures 4 and 5 show a significant improvement in all analysis plots, particularly the DM and Period/P-dot searches. It is noted that whilst the new data SNR is 7.6:1 and the best range SNR is still reported as 8.2:1 but the optimum range has been modified. This can be

expected due to the changed partial folding compression affecting the averaged scintillation and noise result in the compressed sections.

Conclusions

Pulsar feature analysis algorithms have been coded and processed using the SNR statistic to improve detection and verification of weak target acquisitions. The examples show that this has been achieved in this case without any data being removed. As well as demonstrating similarity with the PRESTO confirmation graphic, the present graphic demonstrates close agreement with theoretical data search patterns. A new feature, to combat effects of target scintillation and drift-scan power modulation is ability to scan the recorded data to identify the best range within the data extent to fold for maximum SNR.

The software is made available, still in the development stage, without any guarantees although data from multiple amateur stations has been successfully processed.

References

1. S Ransom, PRESTO, https://www.cv.nrao.edu/~sransom/presto/
2. A Dell'Immagine, G Dell'Immagine, http://iw5bhy.altervista.org/index.php
3. M. Klaassen. 3pt RTL SDR analysis tools, http://parac.eu/projectmk17b.htm
4. A. Dell'Immagine, G. Dell'Immagine. https://github.com/gio54321/pulsar-distro-guide
5. M. Leech. https://github.com/ccera-astro/pulsar_filterbank
6. Osmocom. https://osmocom.org/projects/rtl-sdr/wiki/Rtl-sdr
7. GNURadio: https://www.gnuradio.org/
8. PSRCHIVE, SIGPROC, DSPSR, pulsarastronomy.net/pulsar/software
9. ATNF. https://www.atnf.csiro.au/research/pulsar/psrcat/
10. PW East. Detailed Evaluation of Low SNR Pulsar Data Records, Journal of the Society of Amateur Radio Astronomers., March/April 2022. or, http://www.y1pwe.co.uk/RAProgs/DetailPulsar.pdf
11. PW East. http://www.y1pwe.co.uk/RAProgs/pwepsrsoft.zip
12. Anaconda. https://www.anaconda.com/download
13. PW East http://www.y1pwe.co.uk/RAProgs/PulsarAnalysisLowSNR2.doc

Appendix A1. Determining Pulsar Topocentric Rotation Period: *TopoBary.py*

The professional software TEMPO on the LINUX operating system is the usual way of obtaining the instantaneous value, based on the observation UTC time and the observer's position on the Earth.
For WINDOWS, there is an option to use the free software, ANACONDA.
Within the package is the SPYDER Python IDE and also the ASTROPY add-on module for astronomical calculations. The Anaconda package is also available for MacOS and LINUX.
A python program, *TopoBary.py*, is available to calculate topocentric period specifically for pulsar B0329, but could easily be modified for other pulsars.

The procedure is;-

1. Download and install ANACONDA
2. Copy program TopoBary.py to your working directory
3. Open SPYDER and open TopoBary.py
4. In SPYDER 'run configuration', type observation time, latitude and longitude, as example:

2018-06-27T02:52:00.0 54.21 -1.33

A typical result printed in the terminal is;-

Latt = 54.21 deg
Long = -1.33 deg
UTC Date = 2018-06-27T02:52:00.000
MJD = 58296.11944444444
Doppler Vel = 12227.336479470565 m/s
ppm = -40.78600429457957
B0329 Barycentric period = 0.7145217920645831

B0329 Topocentric period = 0.7144926495757034

The final line indicating the topocentric period in seconds is the figure required to correctly fold B0329 data.

Appendix A2. RTL Data Pre-Processing - *rtlchannel4bin.exe*

Takes the RTL SDR, (*rtl_sdr.exe*) one byte .bin files, takes N-point FFT, splits frequency components and averages in time, ready for folding.
Outputs 4-byte binary file of sequential frequency channelised RTL data at a specified clock rate.

Command format:-

rtlchannel4bin.exe <infile> <outfile> <clock rate(MHz)> <O/P data set rate (kHz)><FFT points, N>

Appendix A3. GNURadio Airspy R2 Data Acquisition and Pre-processing Example

GNURadio is a free, easy to use, graphical package for all receiver digital signal pre-processing needs. There are both Windows and Linux versions that are well-documented.
Figure 9 shows the essential functions of the processing run in the GNU Radio Companion graphical interface. This is a copy extracted from the full filterbank version by Marcus Leech from Reference 5.

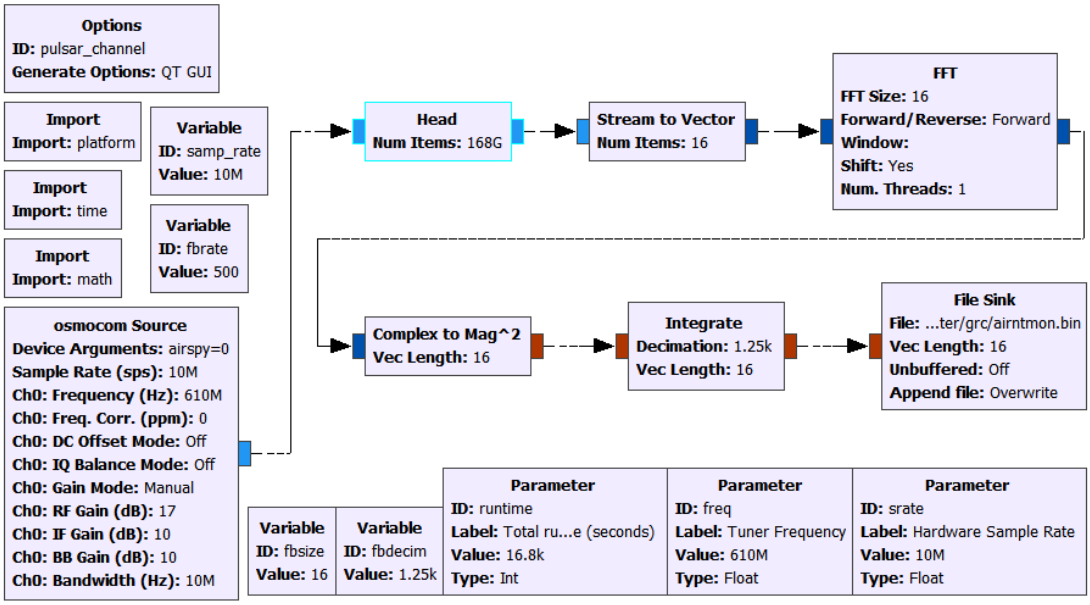

Figure A3 GNURadio: *pulsar_channel_sdr.grc* Graphical View

In the plot, the 'Head' block defines the number of complex data samples processed in the observation. 'Stream to Vector' splits the data, in this example, into 16 complex streams ready for the 16-bit 'FFT' process. 16 complex FFT outputs are then converted to 32-bit floating point magnitudes proportional to the input power in the 16 RF bands.
The 'Integrate' block sums the data in 1250 sample blocks to downsize the data to 16 x 500 sps rate for recording in the 'File Sink' block.

Appendix A4. Pulsar Detection and Analysis - *pulsar_det_an.exe*

Takes ***rtlchannel4bin.exe*** or GNURadio channelised and downsampled pulsar binary data file, partially folds, conditions, matched filters, de-disperses and analyses pulsar matching features and outputs multiple text files suitable for python ***pul_plot.py*** presentation

Command Format:

pulsar_det_an.exe <binfile> <N-point FFT> <data clock (ms))> <ATNF period (ms)> <No: fold sections><No. FFT bins><ATNF pulse width><ATNF DM><ppm><Threshold sigma><ppm range factor><RF band (MHz)><RF Centre (MHz)><roll average No.><start section><end section>

Appendix A5. Partial Folding

Instead of folding all the data into one (M=1) period, as with the normal, standard pulsar data folding, to maximise the pulsar signal-to-noise ratio, the data is partially folded into 'M', typically 128, *synchronous serial* sections of around, say R (nmax/P), periods (P) each as indicated in Figure A6.1. Adding all 128 section periods in parallel gives the same result as folding all the data into one period. All the pulsar detection validation processes can be carried out on this reduced data set much faster and without losing any pulsar wanted information.

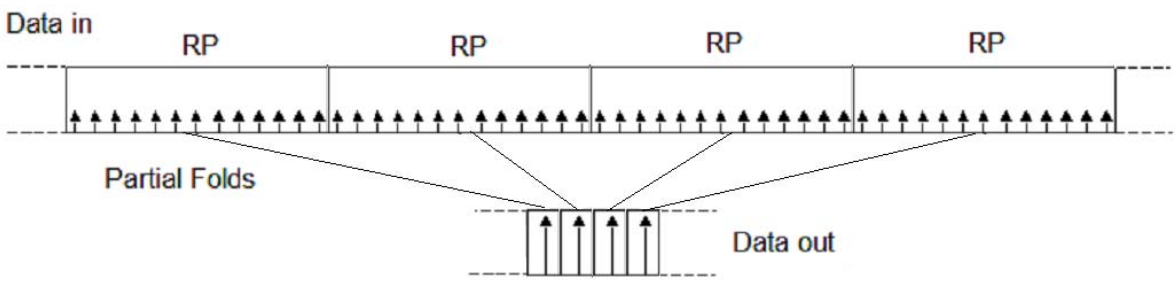

Figure A5.1 Partial Folding Principle

In the following outline guide folding programs,

B is the number of fold bins
M is the number of periods in a partial fold
P is the pulsar period (sample rate assumed 1 ms and period in ms)
Dat is the detected data file
Min/Max are the file sample start/offset and end data numbers

```
Stfold(B,M,P,Dat,Min,Max) := | for fs ∈ 0..M·B − 1
                             |   | bdat_fs ← 0
                             |   | bcount_fs ← 0
                             | for x ∈ Min..Max − 1
                             |   | s ← floor[[(x/(M·P) − floor(x/(M·P)))·M·B] + .5]
                             |   | bdat_s ← bdat_s + Dat_x
                             |   | bcount_s ← bcount_s + 1
                             | for b ∈ 0..M·B − 1
                             |   bindat_b ← bdat_b / bcount_b
                             | bindat
```

```
Sefold(B,M,P,Dat,Max) := | nmax ← floor(Max/M)
                         | for a ∈ 0..B·M − 1
                         |   outm_a ← 0
                         | for x ∈ 0..M − 1
                         |   | xs ← floor(floor(x·nmax/P)·P + .5)
                         |   | for y ∈ xs..xs + nmax − 1
                         |   |   datn_(y−xs) ← Dat_y
                         |   | out ← Stfold(B,1,P,datn,0,nmax)
                         |   | for y ∈ 0..B − 1
                         |   |   outm_(y+B·x) ← out_y
                         | outm
```

Figure A5.2 Standard Fold (left) and Partial/Serial Fold (right) Algorithms

(Note: R = nmax= integer part of Max/M)

Appendix A6 Best Sequence Search Process

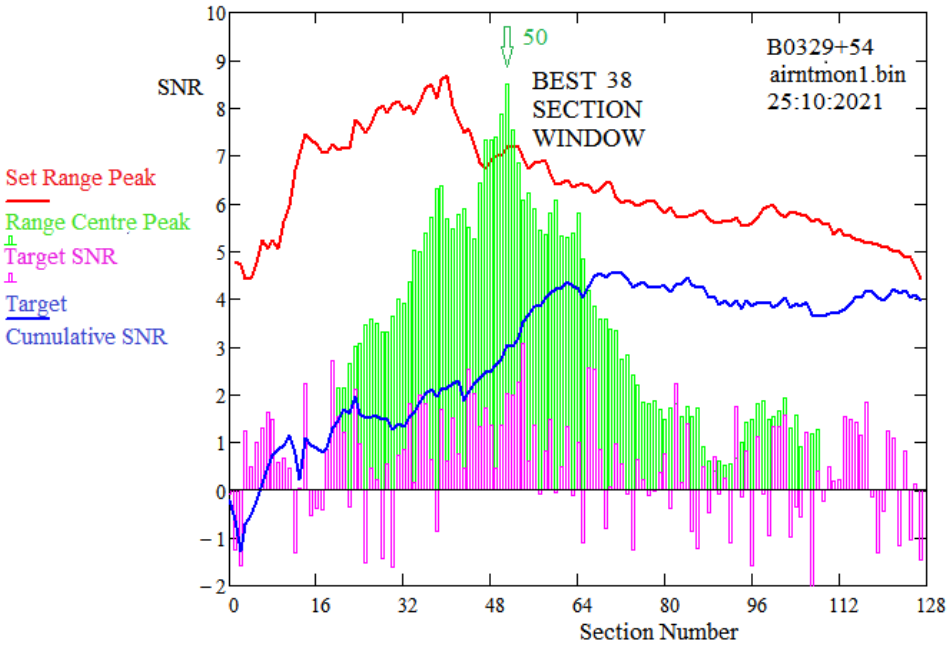

Figure A6.1 Rolling Window Average

The first stage is to locate the target pulsar pulse in the period frame from the fully folded data so as to plot the target SNR in each section SNR, displayed as the magenta plots in Figure A7.1.
They show a concentration around section number 48.
The blue plot shows the cumulative target SNR and this is interesting as it shows a significant correlated rise around section 48; not much change above section 64 although a steady fall in SNR is evident with increasing sections.
The best range can now be found by setting a window of a set number of sections and scanning (rolling) this window over the whole section range and plotting the maximum value it finds.
This is done with all possible window sizes, to produce the red curve.
This shows that just using 40 sections the SNR peaks around 8:1.
The next step takes just 40 adjacent sections and scans these over the range, again plotting the block maximum value and produces the green plots which appears to peak when the 40 sections are centred on section 51.
This means that summing just the sections 31 to 71 would produce the final SNR of 8:1.
Another important conclusion can be drawn from the green plot and that is that the strongest target signal density follows the drift scan antenna pattern, hinting at observing an extra-terrestrial source.

Appendix A7 Compiling C programs to handle large GB files

It is important that .exe programs intended to process multi-GB files are compiled to process these large numbers. Using the gcc compiler on either Windows or Linux, on a 32-bit computer, follow the FileTrim.c example below,

```
C:\Users/Working >gcc FileTrim.c -o FileTrim.exe -lm -D_FILE_OFFSET_BITS=64
```

Appendix A8. Pulsar Data Display - *pul_plot.py*

Python 3 program, ***pul_plot.py***, run in SPYDER, takes ***pulsar_det_an*** output data files (listed in the table below) located in the working directory and presents a .png image (as Figures 4 and 5) of the pulsar analysed data to enable acquisition verification.

Table A8 ***pulsar_det_an.exe*** output data file list used by ***pul_plot.py***

blankf.txt	Frequency channels to attenuate
blanks.txt	Fold sections to attenuate
obsdata.bin	pre-processed observation data for analysis
profile.txt	pulse profile + peak rolling average profile
dmSearch.txt	DM search scan
dmSrchns.txt	DM search with pulsar target blanked
periodS.txt	period search scan
pdotS.txt	pdot search scan
bandS.txt.	band search max scan
bandat.txt	band folds
bandcum.txt	cumulative band folds
cumbands.txt	cumulative SNR of band search
cumsec.txt	cumulative SNR of section search
secsnr.txt	SNR of individual sections
secavsnr.txt	rolling average section SNR
puldat.txt	individual section SNR folds
secavfol.txt	rolling average section folds file
allbands.txt	partial folded data bands combined
spallbands.txt	partial folded video spectrum - bands combined
foldat.txt	cumulative SNR of bins
foldatd.txt	cumulative SNR of bins, de-dispersed SNR
foldatdd.txt	cumulative SNR of bins, DM de-dispersed SNR
ppd2d.txt	2D period/p-dot plot
outdat.txt	matched-filtered output data - continuous stream
rawdat.txt	raw compressed partial folded data
cutdat.bin	data file cut between start and end sections
dmprf.txt	Optimum DM - best profile
secavrol.txt	Section variable block average fold peak
header.txt	Header for Python program
max.txt	maximum bin and SNR
pfold.txt	period search folds
puldatd.txt	individual section Dispersed SNR folds
dmfold.txt	DM search folds
spectrum.txt	input data FFT RF spectrum
data.txt	command line settings

Appendix A9 Software Instruction Summary - ReadMe.txt

Amateur's Alternative to PRESTO for Pulsar Analysis

Two software routes, A and B plus C - Handy programs:-

A. RTL2832U SDR

rtl_sdr.exe + retchannel4bin.exe + pulsar_det_an.exe + pul_plot.py (TEMPO, Topobaryt2.py for topocentric site/date period)

Command Terminal Lines:

a) Data Recording

rtl_sdr.exe ./datafile.bin -g <gain> -f<RF Centre> -s<Sample rate>

Example: rtl_sdr.exe ./datafile.bin -g 40 -f 400e6 -s 2e6

b) Pre-processing

rtlchannel4bin.exe <infile> <outfile> <clock rate(MHz)> <O/P data set rate (khz)><FFT points, N>

Example: rtlchannel4bin.exe datafile.bin outfile.bin 2 1 16

c) Topocentric period - run in Python IDE

Topobary.py <observation time> < latitude> < longitude>

Example: Topobary.py 2018-06-27T02:52:00.0 54.21 -1.33

d) Detection and Analysis

pulsar_det_an.exe <binfile> <N-point FFT> <data clock (ms))> <Topocentric period (ms)> <No: fold sections><No. FFT bins><ATNF pulse width><ATNF DM><ppm offset><threshold sigma><ppm range factor><RF band (MHz)><RF Center (MHz)><roll average No.><start section><end section>

Example: pulsar_det_an.exe outfile.bin 16 1 714.50000 128 1024 6.5 26.7 6 2 2 40 35 0 127

e) Analysis Data Result - run in Python IDE, run in the directory/folder containing pulsar_det_an.exe output files.

Example: pul_plot.py

B. RTL2832U SDR or Airspy or other compatible SDRs

GNURadio (pulsar_channel_sdr.grc)+ pulsar_det_an.exe + pul_plot.py (TEMPO, Topobary.py for topocentric site/date period)

a) Data Recording and Pre-processing

GNURadio IDE + pulsar_channel_sdr.grc

Example: Choose 'Osmocom Source' SDR, Update Variables and Parameters, Update 'File Sink' outdat.bin

b) Topocentric period - run in Python IDE

Topobary.py <observation time> < latitude> < longitude>

Example: Topobary.py 2018-06-27T02:52:00.0 54.21 -1.33

c) Detection and Analysis

pulsar_det_an.exe <binfile> <N-point FFT> <data clock (ms))> <Topocentric period (ms)> <No: fold sections><No. FFT bins><ATNF pulse width><ATNF DM><ppm offset><threshold sigma><ppm range factor><RF band (MHz)><RF Centre (MHz)><roll average No.><start section><end section>

Example: pulsar_det_an.exe outdat.bin 16 1 714.50000 128 1024 6.5 26.7 6 2 2 40 35 0 127

d) Analysis Data Result - run in Python IDE, run in the directory/folder containing pulsar_det_an.exe output files.

Example pul_plot.py

Note: Compiling .c programs to handle large GB files

It is important that .exe programs intended to process multi-GB files are compiled to process these large numbers. Using the gcc compiler on either Windows or Linux, on a 32-bit computer, follow the FileTrim.c example below,

gcc FileTrim.c -o FileTrim.exe -lm -D_FILE_OFFSET_BITS=64

For more information: see, 'An Amateur's Alternative to PRESTO for Pulsar Analysis'

C: Handy Programs:-

1: FileTrim.c -file slicing

Takes .bin files, cuts bytes from front and end of files: OK for 1,2 and 4 byte .bin files check file byte size first.

*Command format:- filetrim <infile> <outfile> <Start Bytes to cut> <End Bytes to cut> */*
Example: obs_20251003.bin obs_20251003cut.bin 0 5560000000

2: rafft22Lg.c -spectrum plotting
Takes rtlsdr bin files applies FFT in blocks and averages over the input data length, plots data to log base 10.

Command format:- rafft22Lg <infile> <outfile><No FFT points>/*
Example command line: obscut.bin obscutfft.txt 4096

3: rapulsar2con.c -all data optimum fold
Takes rtlsdr .bin files, applies averaging and folding algorithm in blocks. Outputs text file of full data matched bandwidth folded average.

Command format:- rapulsar2con <infile> <outfile> <clock rate(MHz)) <No. output data points> <Pulsar period(ms)>< pulsar pulse width>
Example command line: try313.bin try313.txt 2.4 1024 714.482438 6.5